NYU 이벤트 캘린더 — DB 스키마 & API 명세

Oct 3, 2026 · @Keonsang

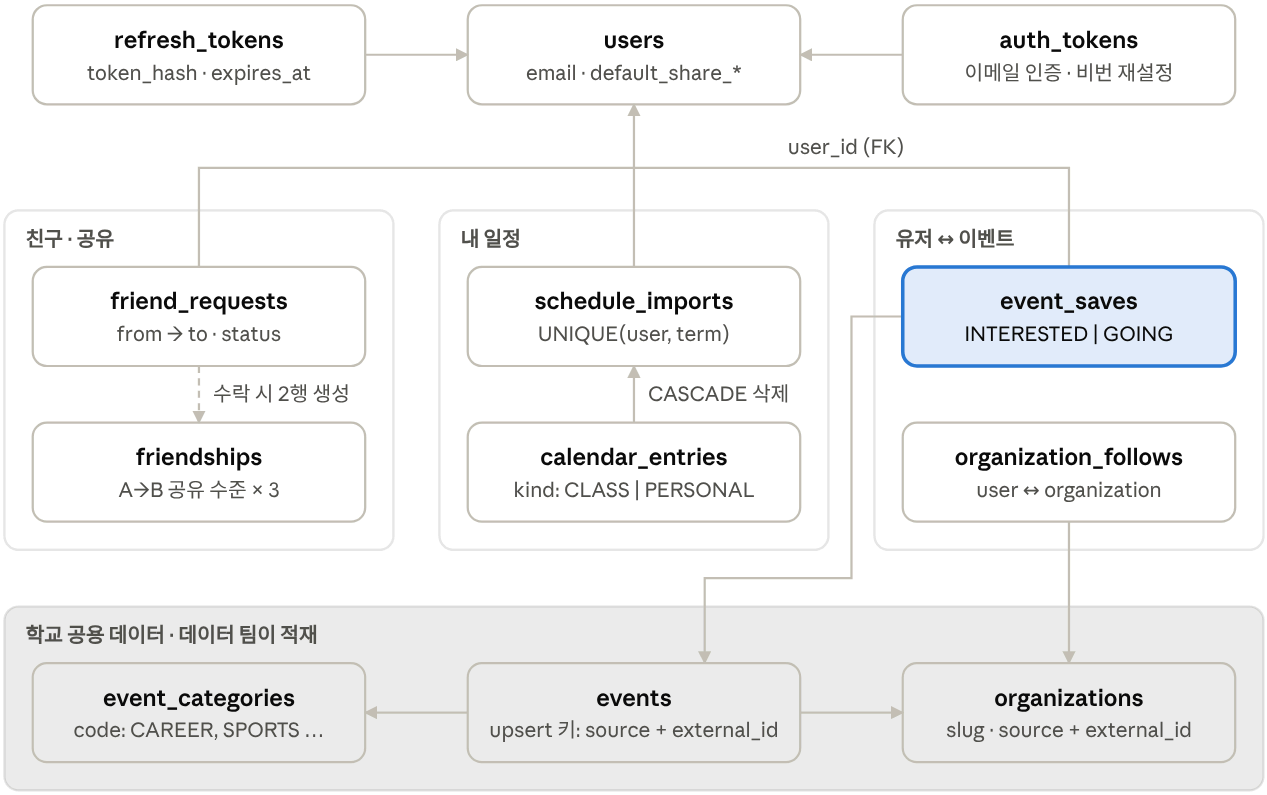
개요

학교 이벤트, Albert 수업 시간표, 개인 일정을 하나의 캘린더로 합쳐 보여주고, 친구에게 친구별·항목 종류별로 공유하는 구조다. 테이블은 12개이며 전체 DDL은 함께 전달한 `schema.sql`(MySQL 8)에 있다.

- **스택**: React + FastAPI + MySQL 8 (InnoDB, utf8mb4). API 경로는 모두 `/api/v1` 아래.
- **인증**: 이메일 + 비밀번호. 짧은 access JWT + 회전되는 refresh 토큰.
- **시간**: DB에는 전부 UTC `DATETIME`으로 저장하고 API도 UTC ISO 8601로 주고받는다. America/New_York 변환은 프론트에서.
- **두 종류의 일정 데이터**: 학교 이벤트(`events`)는 모두가 보는 공용 데이터, 수업·개인 일정(`calendar_entries`)은 유저 소유 데이터로 분리.
- **이벤트**: 카테고리 1개, 반복 이벤트 없음(회차마다 1행). 데이터 팀이 `(source, external_id)` 키로 적재.
- **시간표**: .ics 업로드 시 반복 수업을 회차별 일정으로 펼쳐 저장. 같은 학기를 다시 올리면 덮어쓴다.
- **공유**: 친구 요청 → 수락. 친구별로 수업/개인 일정/저장한 이벤트를 각각 숨김·바쁨·전체 공개 중 하나로 설정.

테이블 관계 (ERD)

유저 데이터와 공용 이벤트는 event_saves에서 만난다



ERD · 12개 테이블, 화살표 = FK가 가리키는 방향

위쪽 세 영역은 유저 소유 데이터라 유저 탈퇴 시 전부 CASCADE 삭제되고, 아래 공용 데이터는 데이터 팀만 쓴다. 두 쪽을 잇는 건 `event_saves`와 `organization_follows` 두 테이블뿐이다. 그림에서 생략한 calendar_entries, friendships, organization_follows의 user_id도 모두 users를 참조한다.

테이블 명세

events (데이터 팀 크롤링 기준)

크롤러는 아래 컬럼을 채우면 된다. 같은 이벤트를 다시 긁으면 `(source, external_id)`로 찾아 덮어쓴다(upsert).

컬럼	타입	필수	설명 / 크롤링 규칙
id	BIGINT UNSIGNED	자동	PK
source	VARCHAR(50)	O	출처 코드. 예: `ENGAGE`, `NYU_EVENTS`, `MANUAL`
external_id	VARCHAR(255)	크롤링 시 O	출처 사이트의 이벤트 ID. 없으면 source_url로 대체
source_url	VARCHAR(500)		원문 페이지 URL
title	VARCHAR(255)	O	이벤트 제목
description	TEXT		본문(HTML 태그 제거한 텍스트)
category_id	TINYINT UNSIGNED	O	`event_categories` FK. 크롤러는 코드(`CAREER` 등)로 매핑
organization_id	BIGINT UNSIGNED		주최 단체 FK. 매칭 안 되면 NULL
host_name	VARCHAR(200)		원문에 적힌 주최자 이름 그대로
start_at	DATETIME	O	시작 시각, **UTC로 변환해서** 저장
end_at	DATETIME	O	종료 시각(UTC). 원문에 없으면 start_at + 1시간
is_all_day	BOOLEAN	O	종일 이벤트면 true (start/end는 뉴욕 자정 기준 UTC)
location_name	VARCHAR(255)		예: `Kimmel Center, Room 914`
address	VARCHAR(255)		도로명 주소
is_online	BOOLEAN	O	온라인 이벤트 여부
online_url	VARCHAR(500)		Zoom 등 링크
registration_url	VARCHAR(500)		RSVP·신청 링크
image_url	VARCHAR(500)		포스터·썸네일
is_free	BOOLEAN		무료 여부, 모르면 NULL
price_text	VARCHAR(100)		예: `$10 / free w/ NYU ID`

컬럼	타입	필수	설명 / 크롤링 규칙
audience_text	VARCHAR(255)		예: `NYU students only`
status	ENUM	O	`ACTIVE` · `CANCELLED` · `HIDDEN`, 기본 ACTIVE
last_synced_at	DATETIME		크롤러가 마지막으로 확인한 시각
created_at / updated_at	DATETIME	자동	

카테고리 초기값: `SOCIAL`, `CAREER`, `SPORTS`, `ACADEMIC`, `ARTS`, `WELLNESS`, `OTHER`.

나머지 테이블

테이블	역할	핵심 컬럼 / 제약
users	계정	email UNIQUE, password_hash, display_name, role(USER/ADMIN), default_share_class·personal·event
refresh_tokens	로그인 세션	token_hash UNIQUE(SHA-256), expires_at, revoked_at
auth_tokens	이메일 인증·비번 재설정	purpose, token_hash, expires_at, used_at
event_categories	고정 카테고리	code UNIQUE, name, color
organizations	동아리·단체	slug UNIQUE, (source, external_id) UNIQUE
organization_follows	단체 팔로우	PK(user_id, organization_id)
event_saves	이벤트 저장 = 내 캘린더에 표시	PK(user_id, event_id), status(INTERESTED/GOING)
calendar_entries	수업 + 개인 일정	kind(CLASS/PERSONAL), start_at, end_at, schedule_import_id, course_code, visibility_override
schedule_imports	Albert 업로드 기록	UNIQUE(user_id, term_code), 삭제 시 해당 수업 일정 CASCADE 삭제
friend_requests	친구 요청	UNIQUE(from_user_id, to_user_id), status

테이블	역할	핵심 컬럼 / 제약
friendships	친구 관계 + 공유 설정	PK(user_id, friend_id), share_class·personal·event

공유 규칙

친구가 내 일정을 얼마나 보는지는 `friendships` 행 하나로 결정된다. 행 (A, B)는 "A가 B에게 공유하는 수준"이고, 친구가 되면 양방향 두 행이 생기며 각자의 기본값이 복사된다.

공유 수준	친구에게 보이는 것
HIDDEN	아무것도 안 보임
BUSY	시간대만 `{"type": "BUSY", start_at, end_at}`. 제목·장소·ID 없음
DETAILS	제목, 장소, 시간 전부

항목 종류	적용 컬럼	기본값
수업 (CLASS)	share_class	DETAILS
개인 일정 (PERSONAL)	share_personal	BUSY
저장한 이벤트	share_event	DETAILS

1. **실제 적용 수준** = min(친구별 설정, 항목의 `visibility_override`). 순서는 HIDDEN < BUSY < DETAILS. 즉 일정 하나를 개별로 더 숨길 수는 있지만 친구 설정보다 더 공개할 수는 없다.
2. **기본값 변경**은 새로 생기는 친구에게만 적용된다. `apply_to_existing_friends: true`를 주면 기존 친구 행도 일괄 갱신.
3. **"친구 N명이 가요"**: 이벤트 X의 친구 목록에는 그 이벤트를 저장했고, 나에게 `share_event = DETAILS`로 공유하는 친구만 포함된다. BUSY로 공유하는 친구는 숫자에도 안 잡힌다.
4. 친구가 아니면 캘린더 조회 자체가 403. 친구를 끊으면 두 행 모두 삭제된다.

API 공통 규칙

- **인증**: `Authorization: Bearer <access_token>` (유효 15분). refresh 토큰(유효 14일)은 HttpOnly·Secure 쿠키로 주고, `/auth/refresh` 호출 때마다 새 토큰으로 교체(회전)한다. DB에는 토큰 원문이 아니라 SHA-256 해시만 저장.
- **가입 제한**: `@nyu.edu` 이메일만 가입 가능, 이메일 인증 전에는 로그인 불가.
- **시간**: 요청·응답 모두 ISO 8601. 응답은 UTC(`2026-10-08T22:00:00Z`), 요청은 오프셋 포함이면 아무 타임존이나 허용.
- **기간 조회**: `start` , `end` 필수, 최대 92일. 조건은 겹침 기준 `start_at < end AND end_at > start` .
- **페이지네이션**(목록형): `?limit=20&cursor=<opaque>` → `{ "items": [...], "next_cursor": "..." | null }` . limit 최대 100.
- **에러 형식**: `{ "error": { "code": "EVENT_NOT_FOUND", "message": "..." } }` . 400 입력 오류, 401 인증 실패, 403 권한 없음, 404 없음, 409 중복, 422 검증 실패.

캘린더 조회 응답의 항목 하나는 아래 모양으로 통일한다. 프론트는 `type` 만 보고 색·아이콘을 고르면 된다.

```
{
  "type": "EVENT",            // CLASS | PERSONAL | EVENT | BUSY
  "id": 1042,                 // BUSY면 없음
  "title": "Fall Career Fair",
  "start_at": "2026-10-08T18:00:00Z",
  "end_at": "2026-10-08T21:00:00Z",
  "is_all_day": false,
  "location": "Kimmel Center",
  "category": "CAREER",       // EVENT만
  "save_status": "GOING",     // EVENT만
  "is_cancelled": false        // EVENT만
}
```

API 엔드포인트

총 54개. 표시가 없으면 로그인 필요, `공개` 는 토큰 없이 호출 가능, `ADMIN` 은 role=ADMIN만.

인증 `/auth`

메서드	경로	설명
POST	/auth/signup	`공개` 가입(email, password, display_name). 인증 메일 발송
POST	/auth/verify-email	`공개` 메일의 토큰으로 인증 완료
POST	/auth/verify-email/resend	`공개` 인증 메일 재발송
POST	/auth/login	`공개` access 토큰 반환 + refresh 쿠키 설정
POST	/auth/refresh	`공개` refresh 쿠키로 토큰 재발급(회전)
POST	/auth/logout	refresh 토큰 폐기
POST	/auth/password/forgot	`공개` 재설정 메일 발송(존재 여부와 무관하게 200)
POST	/auth/password/reset	`공개` 토큰 + 새 비밀번호, 모든 refresh 토큰 폐기

유저 `/users`

메서드	경로	설명
GET	/users/me	내 정보
PATCH	/users/me	display_name, profile_image_url 수정
PUT	/users/me/password	현재 비번 + 새 비번
DELETE	/users/me	탈퇴(관련 데이터 CASCADE 삭제)
GET	/users/me/sharing-defaults	새 친구에게 적용할 기본 공유 수준
PATCH	/users/me/sharing-defaults	`{class?, personal?, event?, apply_to_existing_friends?}`
GET	/users/me/saved-events	저장한 이벤트 목록. `?status=GOING&upcoming=true`
GET	/users/me/following	팔로우한 단체 목록
GET	/users/search	`?q=` 이메일 정확 일치 또는 이름 부분 일치, 최대 20명

메서드	경로	설명
GET	/users/{user_id}	프로필 + friend_status(NONE / REQUESTED / RECEIVED / FRIENDS)

캘린더 `/calendar`

메서드	경로	설명
GET	/calendar	내 통합 캘린더. `?start&end&include=class,personal,event`
GET	/users/{user_id}/calendar	친구 캘린더, 공유 규칙 적용. `?start&end`
GET	/calendar/overlay	나 + 친구 여러 명 겹쳐 보기. `?user_ids=12,31&start&end`, 응답은 user_id별 묶음

내 일정 `/entries` (수업 + 개인)

메서드	경로	설명
POST	/entries	개인 일정 생성(kind는 항상 PERSONAL)
GET	/entries/{entry_id}	상세
PATCH	/entries/{entry_id}	개인 일정은 전 필드, 수업은 description·visibility_override만
DELETE	/entries/{entry_id}	삭제. 수업도 회차 하나 삭제 가능(휴강 등)

Albert 시간표 `/schedule-imports`

메서드	경로	설명
POST	/schedule-imports	multipart `file` (.ics), `term_code?`, `?dry_run=true` 면 미리보기만
GET	/schedule-imports	업로드한 학기 목록
DELETE	/schedule-imports/{import_id}	해당 학기 수업 일정 전체 삭제

이벤트 `/events`

메서드	경로	설명
GET	/events	`공개` 탐색. `?start&end&category&organization_id&q&free_only&followed_only&cursor&limit`
GET	/events/{event_id}	`공개` 상세. 로그인 시 my_save_status, friends_going_count, 친구 미리보기 3명 포함
GET	/events/{event_id}/friends	이 이벤트를 저장한 친구 목록(status 포함)
PUT	/events/{event_id}/save	`{status: INTERESTED / GOING}` 저장 또는 변경
DELETE	/events/{event_id}/save	저장 취소
GET	/events/friends-activity	친구들이 저장한 다가오는 이벤트, 친구 수 많은 순
GET	/categories	`공개` 카테고리 목록

단체 `/organizations`

메서드	경로	설명
GET	/organizations	`공개` 목록·검색. `?q&cursor&limit`
GET	/organizations/{org_id}	`공개` 상세 + 팔로워 수 + is_following
PUT	/organizations/{org_id}/follow	팔로우

메서드	경로	설명
DELETE	/organizations/{org_id}/follow	언팔로우

친구 `/friends`

메서드	경로	설명
GET	/friends	친구 목록 + 내가 각 친구에게 주는 공유 수준
DELETE	/friends/{user_id}	친구 끊기(양방향 행 삭제)
GET	/friends/requests	`?direction=incoming / outgoing`, PENDING만
POST	/friends/requests	`{to_user_id}`. 상대가 이미 보낸 요청이 있으면 즉시 수락 처리
POST	/friends/requests/{request_id}/accept	수락 → friendships 두 행 생성
POST	/friends/requests/{request_id}/decline	거절
DELETE	/friends/requests/{request_id}	보낸 요청 취소
GET	/friends/{user_id}/sharing	내가 이 친구에게 주는 공유 수준
PATCH	/friends/{user_id}/sharing	`{class?, personal?, event?}`

관리자·데이터 적재 `/admin` (`ADMIN`)

메서드	경로	설명
POST	/admin/events/bulk-upsert	최대 500건, `(source, external_id)` 기준 upsert. 결과로 created / updated / failed 개수
POST	/admin/events	수동 등록(source=MANUAL)
PATCH	/admin/events/{event_id}	수정
DELETE	/admin/events/{event_id}	status=HIDDEN 처리(소프트 삭제, 저장 기록 보존)
POST	/admin/organizations/bulk-upsert	단체 일괄 적재

메서드	경로	설명
PATCH	/admin/organizations/{org_id}	수정

데이터 팀은 bulk-upsert API를 써도 되고 DB에 직접 넣어도 된다. 직접 넣을 때도 `INSERT ... ON DUPLICATE KEY UPDATE`로 같은 유니크 키를 쓰면 결과가 같다.

Albert .ics import 흐름

업로드 한 번이 트랜잭션 하나다. 중간에 실패하면 기존 시간표가 그대로 남는다.

1. **업로드**: `.ics` 파일, 최대 1MB. 확장자와 `BEGIN:VCALENDAR` 헤더를 확인.
2. **파싱**: Python `icalendar`로 VEVENT를 읽고, `recurring-ical-events`로 RRULE을 회차별로 펼친다. EXDATE(휴일 등)도 이 단계에서 빠진다.
3. **시간 변환**: TZID(America/New_York) 기준 시각을 UTC로 변환. 서머타임 경계(11월 초)도 라이브러리가 처리.
4. **필드 매핑**: SUMMARY → title, LOCATION → location, UID → ics_uid, SUMMARY에서 정규식으로 course_code 추출(예: `CSCI-UA 202`).
5. **학기 결정**: `term_code`를 안 보내면 첫 수업 날짜로 추정. 1–5월 `SP`, 6–8월 `SU`, 9–12월 `FA` → 예: `2026FA`.
6. **dry_run=true**: DB에 쓰지 않고 과목 목록·회차 수·기간만 반환. 프론트에서 "이대로 가져올까요?" 확인용.
7. **저장**: 같은 (user, term_code)의 기존 import 삭제(수업 일정 CASCADE 삭제) → schedule_imports 1행 삽입 → calendar_entries 일괄 삽입(kind=CLASS).

한 학기 기준 과목 5개, 주 2회, 15주면 약 150행이라 펼쳐 저장해도 부담이 없다.

미결정 사항 · 추후 확장

- [ ] 실제 Albert .ics 파일 하나로 구조 확인(RRULE 방식인지, 회차별 VEVENT인지). 파서는 둘 다 처리하지만 SUMMARY 형식은 봐야 course_code 정규식을 확정할 수 있다.
- [ ] 가입을 `@nyu.edu`로 제한할지 팀 확인.
- [ ] 데이터 팀과 `source` 코드 목록, 카테고리 매핑 규칙, 종료시간 없는 이벤트 처리(+1시간) 합의.

- 이번 범위에서 뺀 것: 알림·리마인더, 이벤트 댓글, 관리자 대시보드, 이벤트 태그, 반복 이벤트. 모두 기존 테이블을 바꾸지 않고 테이블 추가로 붙일 수 있다.
- `/calendar/overlay` 위에 "친구들과 같이 비는 시간 찾기"를 얹을 수 있다.